

Министерство образования Республики Беларусь

Учреждение образования
«Белорусский государственный университет информатики и
радиоэлектроники»

Кафедра электронных вычислительных машин

Дисциплина: Базы данных

Лабораторная работа №6

РАБОЧАЯ ПРОГРАММА ДЛЯ РАБОТЫ С БАЗОЙ ДАННЫХ

Вариант № 26: СТО

Выполнил
студент группы 150503:
Шарай П.Ю.

Проверила:
Игнатович А.О.

Минск 2024

1 ЦЕЛЬ

Разработать программу для взаимодействия с созданной базой данных, демонстрирующую основные принципы работы с PostgreSQL.

2 ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

Разработать программу на языке Python с использованием библиотек tkinter и psycorg2 для взаимодействия с базой данных PostgreSQL. Программа должна предоставлять интерфейс для выполнения SQL-запросов к базе данных, вывода результатов запросов и обработки ошибок.

Оформить отчёт.

3 ХОД ВЫПОЛНЕНИЯ РАБОТЫ

3.1 Общее описание программы

Для работы с базой данных было разработано приложение на языке Python. В качестве инструментов использовались библиотеки «psycorg2» для подключения к базе данных PostgreSQL и tkinter для создания графического интерфейса пользователя.

Взаимодействие с базой данных осуществлялось с помощью языка SQL.

3.2 Описание работы приложения

Приложение предоставляет пользователю возможность взаимодействия с базой данных PostgreSQL. После запуска приложения пользователь видит главное окно, где доступно поле для ввода SQL-запроса и кнопка для его выполнения. Результат выполнения запроса отображается в текстовом поле ниже.

3.3 Схема работы приложения

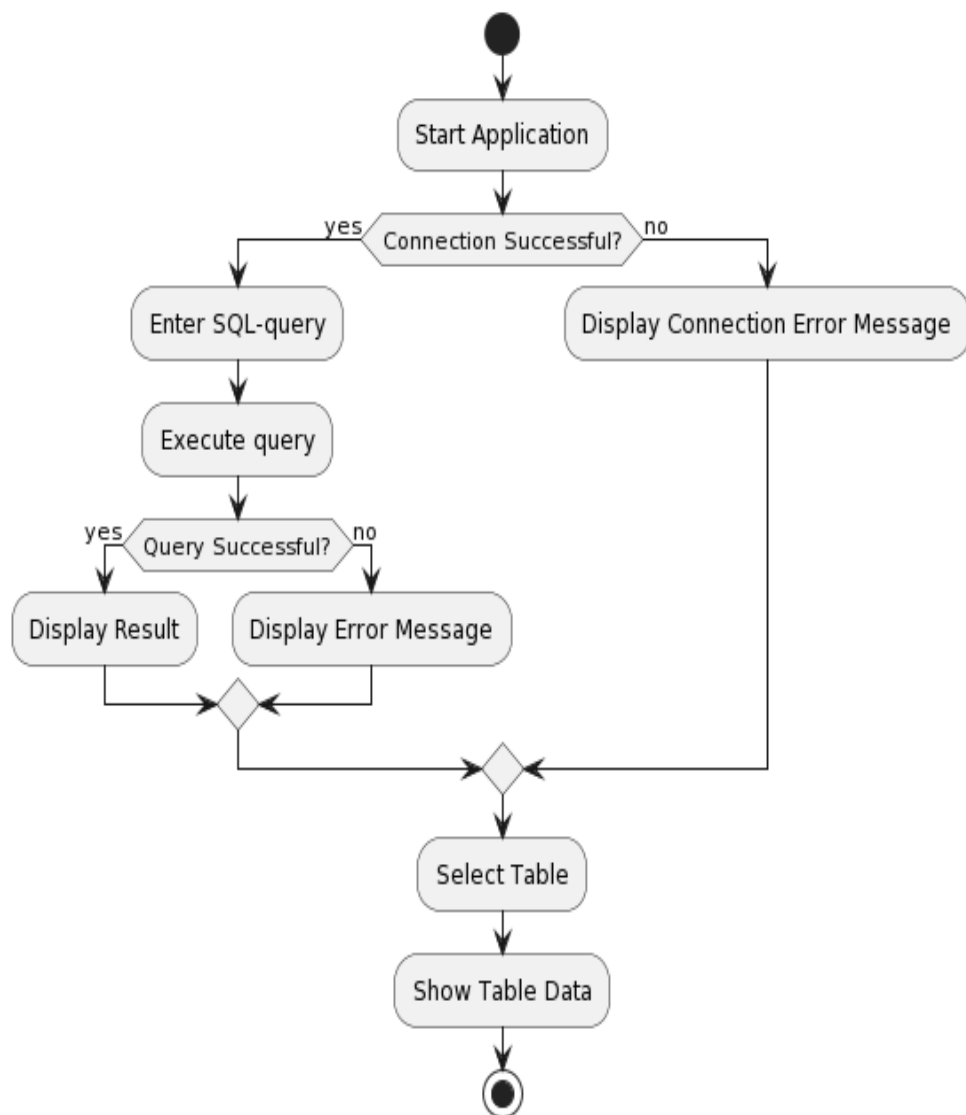


Рисунок 3.3 – Блок-схема работы программы

3.4 Диаграмма классов программы

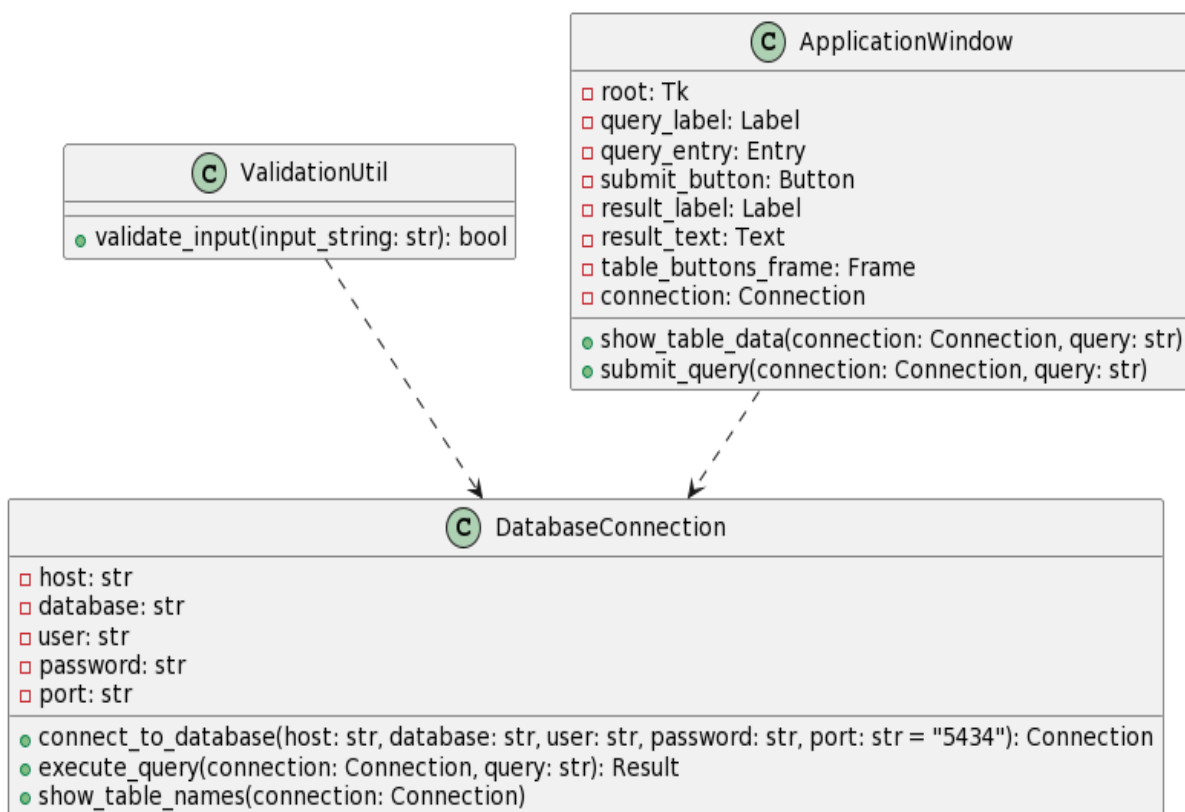


Рисунок 3.4 – Диаграмма классов программы

3.5 Листинг программы

```
import tkinter as tk
from tkinter import messagebox
import psycopg2

def validate_input(input_string):
    """Validate input string."""
    valid_chars =
set('abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789_
-.'')
    return all(char in valid_chars for char in input_string)

def connect_to_database(host, database, user, password,
port="5434"):
    """Функция для подключения к базе данных PostgreSQL."""
    if not all(validate_input(part) for part in (host, database,
user, password)):
        messagebox.showerror("Error", "Wrong symbols.")
        return None
```

```

try:
    connection = psycopg2.connect(
        host="localhost",
        database="STO",
        user="postgres",
        password="password",
        port="5434"
    )
    print("Connection to database is successfull!")
    return connection
except (Exception, psycopg2.Error) as error:
    print(f"Error connection to database: {error}")
    return None

def execute_query(connection, query):
    """Функция для выполнения SQL-запроса к базе данных."""
    try:
        cursor = connection.cursor()
        cursor.execute(query)
        connection.commit()
        print("Query successfully executed!")
        return cursor.fetchall() # Возвращаем результат запроса
    except (Exception, psycopg2.Error) as error:
        print(f"Error executing query: {error}")
        connection.rollback() # Откатываем транзакцию в случае
ОШИБКИ
        return None
    finally:
        cursor.close() # Закрываем курсор после выполнения запроса

def show_table_names(connection):
    cursor = connection.cursor()
    cursor.execute("SELECT table_name FROM information_schema.tables
WHERE table_schema = 'public'")
    table_names = cursor.fetchall()
    cursor.close()

    result_text.delete(1.0, tk.END)
    for table_name in table_names:
        result_text.insert(tk.END, f"{table_name}\n")

# Создание основного окна
root = tk.Tk()
root.title("Database App")

# Создание виджетов
query_label = tk.Label(root, text="Enter SQL-query:")
query_entry = tk.Entry(root, width=50)
submit_button = tk.Button(root, text="Execute query",
command=lambda: submit_query(connection, query_entry.get()))
result_label = tk.Label(root, text="Result:")
result_text = tk.Text(root, width=75, height=30)

# Размещение виджетов на окне
query_label.grid(row=0, column=0, padx=10, pady=5)

```

```

query_entry.grid(row=0, column=1, padx=10, pady=5)
submit_button.grid(row=1, column=0, columnspan=2, padx=10, pady=5)
result_label.grid(row=2, column=0, padx=10, pady=5)
result_text.grid(row=3, column=0, columnspan=2, padx=10, pady=5)

# Подключение к базе данных
connection = connect_to_database("localhost", "STO", "postgres",
"petia2010")

# Создание кнопок для таблиц
table_buttons_frame = tk.Frame(root)
table_buttons_frame.grid(row=4, column=0, columnspan=2, padx=10,
pady=5)

for table_name in ['car', 'client', 'orders', 'service', 'mechanic',
'service_stations', 'order_service']:
    button = tk.Button(table_buttons_frame, text=table_name,
command=lambda t=table_name: show_table_data(connection, f"SELECT *
FROM {t}"))
    button.pack(side=tk.LEFT, padx=5, pady=5)

# Функции для кнопок
def show_table_data(connection, query):
    result = execute_query(connection, query)
    if result is not None:
        result_text.delete(1.0, tk.END)
        # Выводим имена полей таблицы
        cursor = connection.cursor()
        cursor.execute(query)
        field_names = [field[0] for field in cursor.description]
        result_text.insert(tk.END, '\t'.join(field_names))
        result_text.insert(tk.END, '\n')
        cursor.close()
        # Выводим данные
        for row in result:
            result_text.insert(tk.END, '\t'.join(map(str, row)))
            result_text.insert(tk.END, '\n')
    else:
        messagebox.showerror("Error", "Error executing query")

def submit_query(connection, query):
    result = execute_query(connection, query)
    if result is not None:
        result_text.delete(1.0, tk.END)
        # Выводим имена полей таблицы
        cursor = connection.cursor()
        cursor.execute(query)
        field_names = [field[0] for field in cursor.description]
        result_text.insert(tk.END, '\t'.join(field_names))
        result_text.insert(tk.END, '\n')
        cursor.close()
        # Выводим данные
        for row in result:
            result_text.insert(tk.END, '\t'.join(map(str, row)))
            result_text.insert(tk.END, '\n')
    else:

```

```

        messagebox.showerror("Error", "Error executing query")

# Запуск основного цикла приложения
root.mainloop()

```

3.6 Демонстрация работы программы

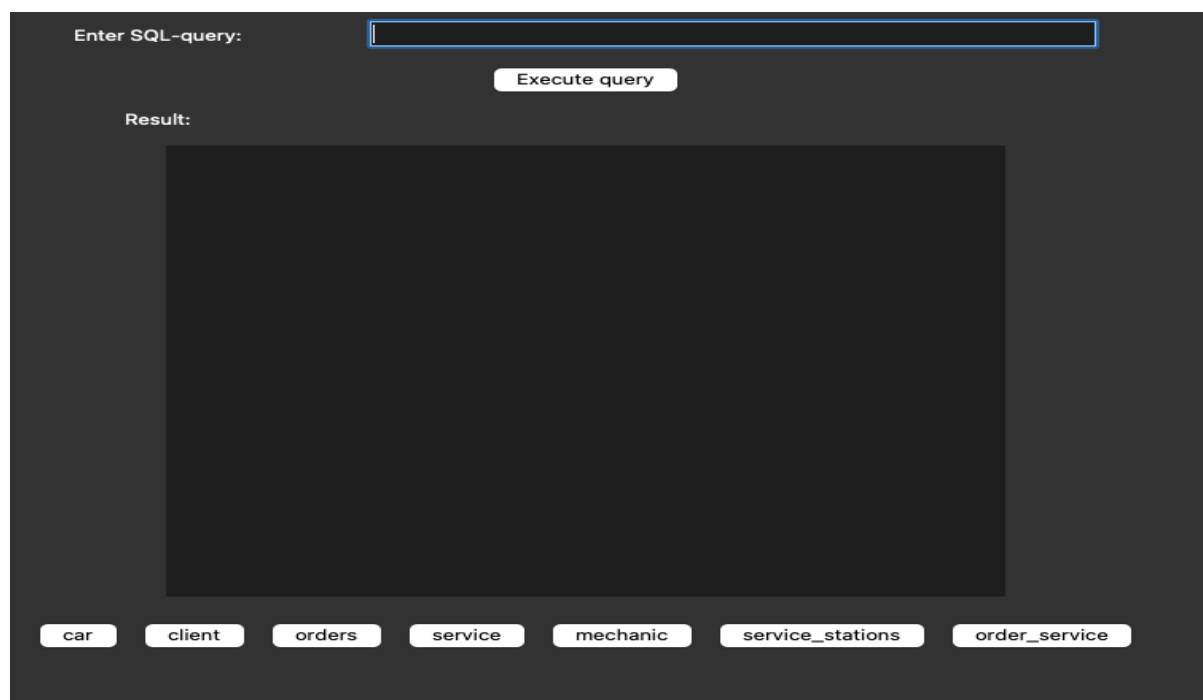


Рисунок 3.6 – Скриншот 1, внешний вид программы

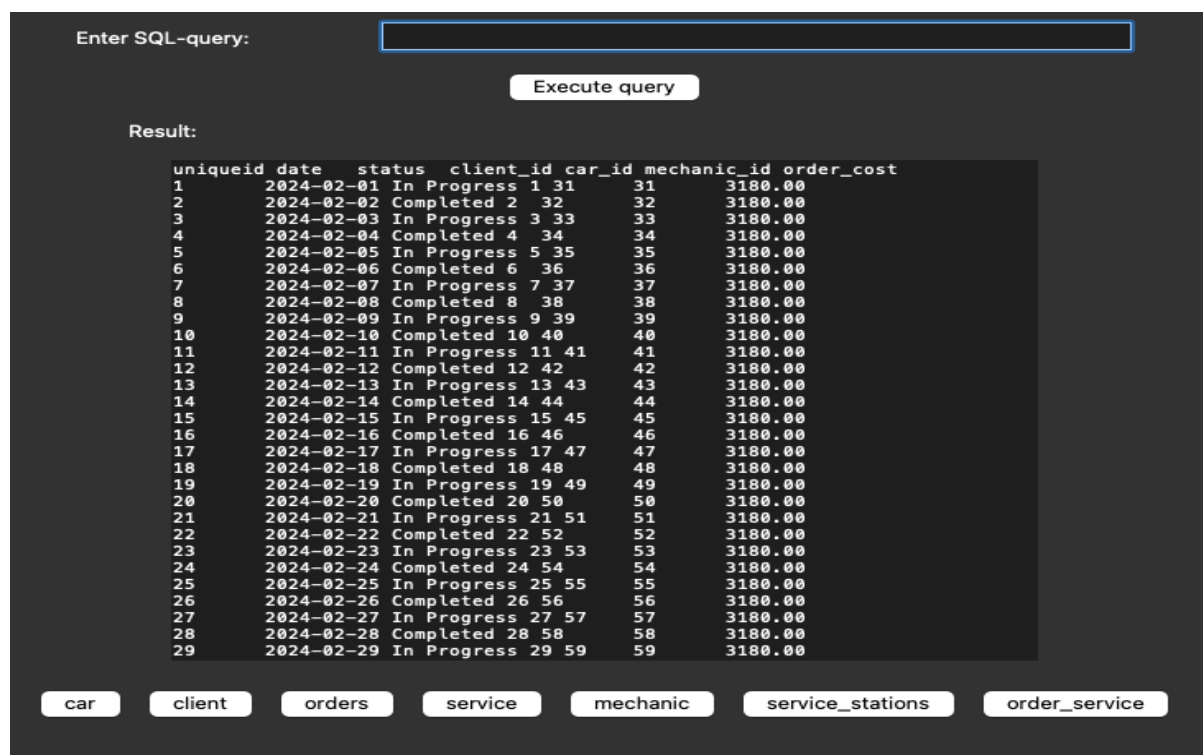


Рисунок 3.6.1 – Скриншот 2, результат работы кнопки «orders»



Рисунок 3.6.2 – Скриншот 3, результат выполнения sql запроса

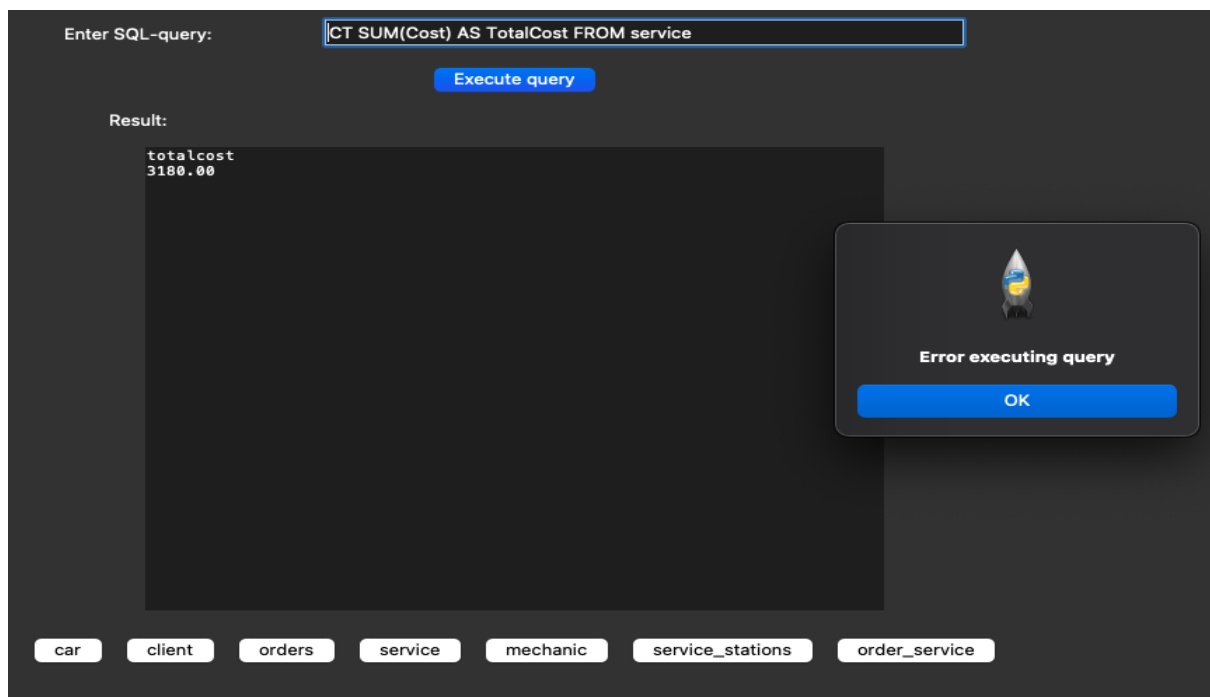


Рисунок 3.6.3 – Скриншот 4, информирование пользователя об ошибке